Subsetting & Vectorization

Lecture 04

Dr. Colin Rundel

Vectorization in R

Vectorized operations in R

Vectorized code applies one operation to many values without writing an explicit loop. R's arithmetic, comparison, and element-wise logical operators are vectorized.

```r
x = 1:5
x^2
```

```
[1]  1  4  9 16 25
```

```r
sqrt(x)
```

```
[1] 1.000000 1.414214 1.732051 2.000000 2.
```

```r
x + 10
```

```
[1] 11 12 13 14 15
```

```r
x > 3
```

```
[1] FALSE FALSE FALSE  TRUE  TRUE
```

```r
x %% 2 == 0
```

```
[1] FALSE  TRUE FALSE  TRUE FALSE
```

```r
(x > 1) & (x < 5)
```

```
[1] FALSE  TRUE  TRUE  TRUE FALSE
```

R recycles by length

When vector lengths differ, R repeats the shorter vector to match the longer one.

```r
x = 1:6
```

```r
x + 10
```

```
[1] 11 12 13 14 15 16
```

```r
x + c(10, 100)
```

```
[1]  11 102  13 104  15 106
```

```r
x + c(10, 100, 1000)
```

```
[1]   11  102 1003   14  105 1006
```

```r
x + c(10, 100, 1000, 10000)
```

```
Warning in x + c(10, 100, 1000, 10000): longer o
of shorter object length
```

```
[1]    11   102  1003 10004    15   106
```

```r
x > 3
```

```
[1] FALSE FALSE FALSE  TRUE  TRUE  TRUE
```

```r
x > c(2, 4)
```

```
[1] FALSE FALSE  TRUE FALSE  TRUE  TRUE
```

```r
x == c(1, 2, 3)
```

```
[1]  TRUE  TRUE  TRUE FALSE FALSE FALSE
```

```r
c(TRUE, FALSE) & c(TRUE, TRUE, TRUE)
```

```
Warning in c(TRUE, FALSE) & c(TRUE, TRUE, TRUE):
multiple of shorter object length
```

```
[1]  TRUE FALSE  TRUE
```

Zero-length recycling

Zero-length vectors contain no values; `NULL` also has length zero. For vectorized operations, the presence of a zero-length vector means the result will also have length 0.

```R
numeric()
```

```
numeric(0)
```

```R
integer()
```

```
integer(0)
```

```R
logical()
```

```
logical(0)
```

```R
character()
```

```
character(0)
```

```R
NULL
```

```
NULL
```

```R
length(NULL)
```

```
[1] 0
```

```R
x = 1:4
```

```R
x + numeric()
```

```
numeric(0)
```

```R
x > numeric()
```

```
logical(0)
```

```R
c(TRUE, FALSE) & logical()
```

```
logical(0)
```

```R
x + NULL
```

```
integer(0)
```

```R
x == NULL
```

```
logical(0)
```

Vectorization in Python

Python lists are not vectorized

Python's list operators manipulate the container, not the values. Element-wise arithmetic and comparisons instead require explicit iteration: a `for` loop or, more idiomatically, a comprehension.

```py
x = [1, 2, 3]
```

```py
x * 2
```

```
[1, 2, 3, 1, 2, 3]
```

```py
x + x
```

```
[1, 2, 3, 1, 2, 3]
```

```py
x + 2
```

```
TypeError: can only concatenate list (not
```

```py
x > 2
```

```
TypeError: '>' not supported between insta
```

```py
x + [2]
```

```
[1, 2, 3, 2]
```

```py
x > [2]
```

```
False
```

List comprehensions

A comprehension combines an expression with a `for` clause to construct a list. A trailing `if` filters values; a conditional expression transforms every value.

```py
result = []
for x in range(6):
    result.append(x**2)
result
```

```
[0, 1, 4, 9, 16, 25]
```

```py
[x**2 for x in range(6)]
```

```
[0, 1, 4, 9, 16, 25]
```

```py
[x**2 for x in range(10)
 if x % 2 == 0]
```

```
[0, 4, 16, 36, 64]
```

```py
[x if x % 2 == 0 else -1
 for x in range(10)]
```

```
[0, -1, 2, -1, 4, -1, 6, -1, 8, -1]
```

Comprehensions can be nested. Additional `for` clauses act like nested loops and produce one flat list, while nested comprehensions build a list of lists.

```py
[(i, j)
 for i in range(3)
 for j in range(2)]
```

```
[(0, 0), (0, 1), (1, 0), (1, 1), (2, 0), (2, 1)]
```

```py
[[i * j for j in range(4)]
 for i in range(3)]
```

```
[[0, 0, 0, 0], [0, 1, 2, 3], [0, 2, 4, 6]]
```

NumPy

NumPy is a third-party Python package for numerical computing. Its main data structure is the homogeneous, multidimensional `ndarray`, with operations designed to work efficiently across entire arrays (i.e. vectorization).

Installation happens once per project (e.g. `uv add numpy`). Before using NumPy in a Python session or script, import it; `np` is the conventional short alias, so functions are called as `np.array()`, `np.arange()`, `np.sqrt()`, etc.

```py
import numpy as np
np.__version__
```

```
'2.5.2'
```

Vectorized operations with NumPy

NumPy arrays support the same style of vectorized arithmetic, comparison, and logical operations as R's atomic vectors, with a similar performance advantage over explicit `for` loops.

```py
x = np.arange(1, 6)
x**2
```

```
array([ 1,  4,  9, 16, 25])
```

```py
np.sqrt(x)
```

```
array([1.        , 1.41421356, 1.73205081,
```

```py
x + 10
```

```
array([11, 12, 13, 14, 15])
```

```py
x > 3
```

```
array([False, False, False,  True,  True])
```

```py
x % 2 == 0
```

```
array([False,  True, False,  True, False])
```

```py
(x > 1) & (x < 5)
```

```
array([False,  True,  True,  True, False])
```

Vectorized conditional choice

`np.where()` is NumPy's equivalent to R's `ifelse()`; both construct a new vector by choosing between two values element-wise based on a condition.

```r
x = 0:7
ifelse(x %% 2 == 0, x, -1)
```

```
[1]  0 -1  2 -1  4 -1  6 -1
```

```py
x = np.arange(8)
np.where(x % 2 == 0, x, -1)
```

```
array([ 0, -1,  2, -1,  4, -1,  6, -1])
```

```r
ifelse(x > 5, "big", "small")
```

```
[1] "small" "small" "small" "small" "small" "sma
```

```py
np.where(x > 5, "big", "small")
```

```
array(['small', 'small', 'small', 'small', 'smal
      dtype='<U5')
```

With a single argument, `np.where()` instead returns the positions where the condition is true; the R equivalent is `which()`.

```r
which(x > 5)
```

```
[1] 7 8
```

```py
np.where(x > 5)
```

```
(array([6, 7]),)
```

The single-argument form returns a tuple with one array of positions per dimension. `np.flatnonzero()` returns a plain

Subsetting

Indexing conventions

R uses 1-based indexing; Python uses 0-based indexing when subsetting by non-negative integers.

```r
x = c("a", "b", "c", "d", "e")
```

```r
x[1]
```

```
[1] "a"
```

```r
x[5]
```

```
[1] "e"
```

```python
x = ["a", "b", "c", "d", "e"]
```

```python
x[0]
```

```
'a'
```

```python
x[4]
```

```
'e'
```

In R, a negative index *excludes* a position. In Python, it counts backward from the end (−1 is the last element).

```r
x[-1]
```

```
[1] "b" "c" "d" "e"
```

```python
x[-1]
```

```
'e'
```

Python slices

A slice has the form `start:stop:step`; `start` is included and `stop` is excluded. Any component may be omitted.

```py
x = list("abcdefgh"); x
```

```
['a', 'b', 'c', 'd', 'e', 'f', 'g', 'h']
```

```py
x[1:5]
```

```
['b', 'c', 'd', 'e']
```

```py
x[:4]
```

```
['a', 'b', 'c', 'd']
```

```py
x[4:]
```

```
['e', 'f', 'g', 'h']
```

```py
x[:]
```

```
['a', 'b', 'c', 'd', 'e', 'f', 'g', 'h']
```

```py
x[1:7:2]
```

```
['b', 'd', 'f']
```

```py
x[::2]
```

```
['a', 'c', 'e', 'g']
```

```py
x[::-1]
```

```
['h', 'g', 'f', 'e', 'd', 'c', 'b', 'a']
```

```py
x[-3:]
```

```
['f', 'g', 'h']
```

Selecting multiple positions

R accepts a vector of positions inside `[ ]`. A base Python list does not: a slice can select a regular range, but arbitrary positions require a comprehension.

```r
x = letters[1:5]
```

```r
x[c(1, 3, 5)]
```

```
[1] "a" "c" "e"
```

```r
x[c(4, 2, 2)]
```

```
[1] "d" "b" "b"
```

```r
x[c(1, -1)]
```

```
Error in `x[c(1, -1)]`:
! only 0's may be mixed with negative subscripts
```

```python
x = list("abcde")
```

```python
x[[0, 2, 4]]
```

```
TypeError: list indices must be integers or slic
```

```python
[x[i] for i in [0, 2, 4]]
```

```
['a', 'c', 'e']
```

```python
[x[i] for i in [3, 1, 1]]
```

```
['d', 'b', 'b']
```

```python
[x[i] for i in [0, -1]]
```

```
['a', 'e']
```

Both languages preserve the requested order and repeated positions. R cannot mix positive and negative indexes in one

Integer array indexing

Unlike base Python lists, NumPy arrays accept a list or integer array of positions. As with list indexing, negative positions count backward from the end and can be mixed with positive positions.

```py
x = np.arange(10, 60, 10); x
```

```
array([10, 20, 30, 40, 50])
```

```py
x[[0, 2, 4]]
```

```
array([10, 30, 50])
```

```py
x[np.array([0, 2, 4])]
```

```
array([10, 30, 50])
```

```py
x[[3, 1, 1]]
```

```
array([40, 20, 20])
```

```py
x[[0, -1]]
```

```
array([10, 50])
```

Selecting with an integer array or list is called *advanced indexing*; selecting with a scalar or slice is *basic indexing*. The

Out-of-bounds positions

Out-of-bounds indexing fails in Python, while R returns a typed missing value for a position that does not exist. Python slices clip each endpoint to the nearest valid boundary.

```r
x = c(10, 20, 30)
```

```r
x[4]
```

```
[1] NA
```

```r
x[c(1, 4)]
```

```
[1] 10 NA
```

```r
x[-4]
```

```
[1] 10 20 30
```

```python
x = [10, 20, 30]
```

```python
x[4]
```

```
IndexError: list index out of range
```

```python
x[:4]
```

```
[10, 20, 30]
```

```python
x[-4:]
```

```
[10, 20, 30]
```

```python
x[4:]
```

```
[]
```

```python
x[:-4]
```

```
[]
```

R - Four additional index types

We have already used positive integers to select positions and negative integers to exclude positions. R's `[` operator supports four additional index forms:

- logicals - select by `TRUE` positions
- characters - select by names
- zero - select nothing
- empty index - select everything

Logical indexes

Logical subsetting keeps values corresponding to TRUE.

Most logical indexes are created by vectorized logical expressions, which return one logical value for each element. Such a vector is often called a *mask*.

```r
x = c(10, 15, 20, 25, 30)
```

```r
x %% 20 == 0
```

```
[1] FALSE FALSE  TRUE FALSE FALSE
```

```r
x[x %% 20 == 0]
```

```
[1] 20
```

```r
(x > 10) & (x < 30)
```

```
[1] FALSE  TRUE  TRUE  TRUE FALSE
```

```r
x[(x > 10) & (x < 30)]
```

```
[1] 15 20 25
```

```r
keep = x > 18
keep
```

```
[1] FALSE FALSE  TRUE  TRUE  TRUE
```

```r
x[keep]
```

```
[1] 20 25 30
```

```r
x[c(TRUE, NA, FALSE, TRUE, FALSE)]
```

```
[1] 10 NA 25
```

NumPy & Boolean indexes

NumPy also supports using Boolean arrays for subsetting - positions where the index is `True` are kept and those with `False` are discarded. Boolean ndarrays, lists of Booleans, and the masks produced by vectorized comparisons all work.

```py
x = np.arange(6)
```

```py
x[np.array([True, False, True, False, True, False])]
```

```
array([0, 2, 4])
```

```py
x[[True, False, True, False, True, False]]
```

```
array([0, 2, 4])
```

```py
x[x % 2 == 0]
```

```
array([0, 2, 4])
```

```py
x[x > 2]
```

```
array([3, 4, 5])
```

However, while R recycles logical vectors, NumPy requires a Boolean index to match the dimension it selects.

```r
x = 0:5
```

```r
x[c(TRUE, FALSE)]
```

```
[1] 0 2 4
```

```py
x[np.array([True, False])]
```

```
IndexError: boolean index did not match indexed
```

```py
x[[True]]
```

```
IndexError: boolean index did not match indexed
```

Boolean expressions with NumPy

NumPy overloads `&`, `|`, and `~` as element-wise logical operators.

Parenthesize every comparison because these operators have higher precedence than comparison operators.

```py
x = np.arange(10)
```

```py
(x > 2) & (x < 7)
```

```
array([False, False, False,  True,  True,
       False])
```

```py
(x < 2) | (x > 7)
```

```
array([ True,  True, False, False, False,
        True])
```

```py
x[(x > 2) & (x < 7)]
```

```
array([3, 4, 5, 6])
```

```py
x[~(x % 2 == 0)]
```

```
array([1, 3, 5, 7, 9])
```

Character indexes

Names allow values to be selected independently of their positions.

```r
x = c(a = 10, b = 20, c = 30)
```

```r
x["a"]
```

```
 a
10
```

```r
x[c("c", "a", "a")]
```

```
 c  a  a
30 10 10
```

```r
x["d"]
```

```
<NA>
  NA
```

```r
x[c("a", "d")]
```

```
   a <NA>
  10   NA
```

```r
unname(x["a"])
```

```
[1] 10
```

Empty and zero indexes

In R, an empty index selects everything and a zero index selects nothing. This differs from Python, where `0` is the first position and `:` is used to select everything.

```r
x = c(10, 20, 30)
```

```r
x[]
```

```
[1] 10 20 30
```

```r
x[0]
```

```
numeric(0)
```

```r
x[NULL]
```

```
numeric(0)
```

```r
x[c(0, 2, 0)]
```

```
[1] 20
```

```py
x = [10, 20, 30]
```

```py
x[:]
```

```
[10, 20, 30]
```

```py
x[0]
```

```
10
```

```py
x[0:0]
```

```
[]
```

In R, zeros may appear alongside positive or negative indexes and are ignored. `NULL` and an empty integer vector also

Subsetting and assignment

Subset syntax can appear on the left side of an assignment in R, base Python, and NumPy. A base Python list supports assignment to a single position or a slice, while R and NumPy also allow assignment via logical / Boolean and integer indexes.

R
```r
x = c(1, 4, 7, 9)
x[2] = 2
x[x %% 2 != 0] = -1
x
```

```
[1] -1  2 -1 -1
```

py
```python
x = [1, 4, 7, 9]
x[1] = 2
x[2:4] = [0, 0, 0]
x
```

```
[1, 2, 0, 0, 0]
```

py
```python
x = np.array([1, 4, 7, 9])
x[1] = 2
x[x % 2 != 0] = -1
x
```

```
array([-1,  2, -1, -1])
```

A slice assignment can change a list's length. NumPy arrays have fixed size and type - assignment can replace values, but

NumPy views and copies

Basic slicing always returns a *view* that shares data with the original array. Advanced indexing returns a *copy*.

```py
x = np.arange(6)
y = x[1:4]
y[0] = 99
x
```

```
array([ 0, 99,  2,  3,  4,  5])
```

```py
np.shares_memory(x, y)
```

```
True
```

```py
x = np.arange(6)
z = x[[1, 2, 3]]
z[0] = 99
x
```

```
array([0, 1, 2, 3, 4, 5])
```

```py
np.shares_memory(x, z)
```

```
False
```

Exercise 1

Without running the code, determine the result (or error) of each expression.

R
```r
x = c(a = 2, b = 4, c = 6, d = 8)
```

py
```python
x = np.array([2, 4, 6, 8])
```

R
```r
x[c(1, 3)]
x[-c(1, 3)]
x[c(TRUE, FALSE)]
x[c("d", "b", "z")]
x[x > 3 & x < 8]
```

py
```python
x[[0, 2]]
x[-2:]
x[np.array([True, False])]
x[x > 3 & x < 8]
x[(x > 3) & (x < 8)]
```

Matrices & arrays

R matrices and NumPy arrays

Both are homogeneous, multidimensional containers designed for vectorized computation.

```r
x = matrix(1:6, nrow = 2, ncol = 3)
x
```

```
     [,1] [,2] [,3]
[1,]    1    3    5
[2,]    2    4    6
```

```r
typeof(x)
```

```
[1] "integer"
```

```r
dim(x)
```

```
[1] 2 3
```

```python
x = np.array([[1, 2, 3],
              [4, 5, 6]])
x
```

```
array([[1, 2, 3],
       [4, 5, 6]])
```

```python
x.dtype
```

```
dtype('int64')
```

```python
x.shape
```

```
(2, 3)
```

Creating arrays

```r
matrix(0, nrow = 2, ncol = 3)
```

```
     [,1] [,2] [,3]
[1,]    0    0    0
[2,]    0    0    0
```

```r
array(1, dim = c(2, 2, 2))
```

```
, , 1

     [,1] [,2]
[1,]    1    1
[2,]    1    1

, , 2

     [,1] [,2]
[1,]    1    1
[2,]    1    1
```

```r
diag(3)
```

```
     [,1] [,2] [,3]
[1,]    1    0    0
[2,]    0    1    0
[3,]    0    0    1
```

```r
seq(0, 1, length.out = 5)
```

```
[1] 0.00 0.25 0.50 0.75 1.00
```

```python
np.zeros((2, 3))
```

```
array([[0., 0., 0.],
       [0., 0., 0.]])
```

```python
np.ones((2, 2, 2))
```

```
array([[[1., 1.],
        [1., 1.]],

       [[1., 1.],
        [1., 1.]]])
```

```python
np.eye(3)
```

```
array([[1., 0., 0.],
       [0., 1., 0.],
       [0., 0., 1.]])
```

```python
np.linspace(0, 1, 5)
```

```
array([0.  , 0.25, 0.5 , 0.75, 1.  ])
```

Column-major vs row-major

R matrices use *column-major* ordering - a sequence of values fills the matrix down each column. NumPy arrays default to *row-major* ordering - values fill across each row.

```r
x = matrix(1:6, nrow = 2)
x
```

```
     [,1] [,2] [,3]
[1,]    1    3    5
[2,]    2    4    6
```

```r
y = matrix(1:6, nrow = 3, byrow = TRUE)
y
```

```
     [,1] [,2]
[1,]    1    2
[2,]    3    4
[3,]    5    6
```

```r
c(y)
```

```
[1] 1 3 5 2 4 6
```

```py
x = np.arange(1, 7).reshape((2, 3))
x
```

```
array([[1, 2, 3],
       [4, 5, 6]])
```

```py
y = np.array([[1, 2],
              [3, 4],
              [5, 6]])
y
```

```
array([[1, 2],
       [3, 4],
       [5, 6]])
```

```py
y.flatten()
```

```
array([1, 2, 3, 4, 5, 6])
```

Two-dimensional indexing

Dimensions are separated by commas in both languages; ranges select rectangular regions.

```r
x = matrix(1:16, nrow = 4, byrow = TRUE); x
```

```
     [,1] [,2] [,3] [,4]
[1,]    1    2    3    4
[2,]    5    6    7    8
[3,]    9   10   11   12
[4,]   13   14   15   16
```

```r
x[1, 2]
```

```
[1] 2
```

```r
x[2:3, 2:4]
```

```
     [,1] [,2] [,3]
[1,]    6    7    8
[2,]   10   11   12
```

```r
x[c(1, 3), c(2, 4)]
```

```
     [,1] [,2]
[1,]    2    4
[2,]   10   12
```

```python
x = np.arange(1, 17).reshape(4, 4); x
```

```
array([[ 1,  2,  3,  4],
       [ 5,  6,  7,  8],
       [ 9, 10, 11, 12],
       [13, 14, 15, 16]])
```

```python
x[0, 1]
```

```
np.int64(2)
```

```python
x[1:3, 1:4]
```

```
array([[ 6,  7,  8],
       [10, 11, 12]])
```

```python
x[np.ix_([0, 2], [1, 3])]
```

```
array([[ 2,  4],
       [10, 12]])
```

Integer arrays in 2D

An integer list can select along either dimension. When both dimensions receive integer lists, NumPy pairs them element by element instead of crossing them.

```py
x = np.arange(1, 17).reshape(4, 4)
x[[0, 2], :]
```

```
array([[ 1,  2,  3,  4],
       [ 9, 10, 11, 12]])
```

```py
x[:, [1, 3]]
```

```
array([[ 2,  4],
       [ 6,  8],
       [10, 12],
       [14, 16]])
```

```py
x[[0, 2], [1, 3]]
```

```
array([ 2, 12])
```

```py
x[np.ix_([0, 2], [1, 3])]
```

```
array([[ 2,  4],
       [10, 12]])
```

The paired expression selects `(row 0, column 1)` and `(row 2, column 3)`, not a 2×2 rectangle; `np.ix_()` produces the rectangle.

Reductions and axes

A *reduction* combines many values into fewer values. NumPy's `axis` argument specifies which dimension is collapsed.

```r
x = matrix(1:12, nrow = 3, byrow = TRU
```

```
     [,1] [,2] [,3] [,4]
[1,]    1    2    3    4
[2,]    5    6    7    8
[3,]    9   10   11   12
```

```r
sum(x)
```

```
[1] 78
```

```r
rowMeans(x)
```

```
[1]  2.5  6.5 10.5
```

```r
colMeans(x)
```

```
[1] 5 6 7 8
```

```python
x = np.arange(1, 13).reshape(3, 4); x
```

```
array([[ 1,  2,  3,  4],
       [ 5,  6,  7,  8],
       [ 9, 10, 11, 12]])
```

```python
x.sum()
```

```
np.int64(78)
```

```python
x.mean(axis=1)
```

```
array([ 2.5,  6.5, 10.5])
```

```python
x.mean(axis=0)
```

```
array([5., 6., 7., 8.])
```

NumPy broadcasting

NumPy broadcasts by shape

R recycles by comparing lengths; NumPy *broadcasts* by comparing shapes.

Shapes are compared from the rightmost dimension to the left, and each pair of dimensions is compatible when they are equal or one is 1. An array with fewer dimensions is treated as if it had leading dimensions of size 1.

```py
x = np.arange(1, 7)
```

```py
x + 10
```

```
array([11, 12, 13, 14, 15, 16])
```

```py
x + np.array([10])
```

```
array([11, 12, 13, 14, 15, 16])
```

```py
x + np.array([10, 100])
```

```
ValueError: operands could not be broadcast toge
```

```py
x + np.array([10, 100, 1000,
              10_000, 100_000, 1_000_000])
```

```
array([     11,     102,    1003,   10004,  1000
```

Adding a vector to every row

A one-dimensional array aligns with the trailing (column) dimension, so it is applied to every row.

```py
x = np.arange(12).reshape(4, 3)
y = np.array([10, 20, 30])
```

```py
x
```

```
array([[ 0,  1,  2],
       [ 3,  4,  5],
       [ 6,  7,  8],
       [ 9, 10, 11]])
```

```py
x.shape
```

```
(4, 3)
```

```py
y
```

```
array([10, 20, 30])
```

```py
y.shape
```

```
(3,)
```

```py
x + y
```

```
array([[10, 21, 32],
       [13, 24, 35],
       [16, 27, 38],
       [19, 30, 41]])
```

```py
(x + y).shape
```

```
(4, 3)
```

Adding a vector to every column

To align with the row dimension instead, give the vector an explicit singleton column dimension.

```py
x = np.arange(12).reshape(3, 4)
y = np.array([10, 20, 30])
```

```py
y.shape
```

```
(3,)
```

```py
y[:, np.newaxis]
```

```
array([[10],
       [20],
       [30]])
```

```py
y[:, np.newaxis].shape
```

```
(3, 1)
```

```py
x + y[:, np.newaxis]
```

```
array([[10, 11, 12, 13],
       [24, 25, 26, 27],
       [38, 39, 40, 41]])
```

Pairwise combinations

Broadcasting two singleton dimensions produces every pairwise combination.

R

```r
x = 1:3
y = c(10, 20, 30, 40)
outer(x, y, FUN = "+")
```

```
     [,1] [,2] [,3] [,4]
[1,]   11   21   31   41
[2,]   12   22   32   42
[3,]   13   23   33   43
```

py

```python
x = np.arange(1, 4)
y = np.arange(10, 50, 10)
x[:, np.newaxis] + y
```

```
array([[11, 21, 31, 41],
       [12, 22, 32, 42],
       [13, 23, 33, 43]])
```

Standardizing columns

Broadcasting makes it possible to transform every column using its own mean and standard deviation.

```py
rng = np.random.default_rng(523)
x = rng.normal(
    loc=[-1, 0, 1],
    scale=[1, 2, 3],
    size=(1000, 3)
)
```

```py
means = x.mean(axis=0)
sds = x.std(axis=0)
means
```

```
array([-1.01363555,  0.06318758,  0.844760
```

```py
sds
```

```
array([0.94966875, 1.94205801, 2.8675822 ]
```

```py
z = (x - means) / sds
z.mean(axis=0)
```

```
array([-4.35207426e-17, -7.28306304e-17,
```

```py
z.std(axis=0)
```

```
array([1., 1., 1.])
```

Unintended broadcasting

Valid shapes do not guarantee the intended calculation: both expressions below run, but only the second subtracts each row's own mean.

```py
x = np.array([[1, 2, 3],
              [4, 5, 6],
              [7, 8, 9]])
row_means = x.mean(axis=1)
```

```py
x - row_means
```

```
array([[-1., -3., -5.],
       [ 2.,  0., -2.],
       [ 5.,  3.,  1.]])
```

```py
x - row_means[:, np.newaxis]
```

```
array([[-1.,  0.,  1.],
       [-1.,  0.,  1.],
       [-1.,  0.,  1.]])
```

When an expression is surprising, inspect `.shape` before the values. `np.broadcast_shapes()` can predict the result shape

Exercise 2

For each pair of NumPy shapes, determine whether broadcasting succeeds and, if so, the result shape.

1. `(128, 128, 3)` and `(3,)`
2. `(8, 1, 6, 1)` and `(7, 1, 5)`
3. `(2, 1)` and `(8, 4, 3)`
4. `(3, 1)` and `(15, 3, 5)`
5. `(3,)` and `(4,)`

Then write a vectorized NumPy expression that subtracts the mean of each row from a matrix `x` with shape `(100, 5)`.

Comparing R & Python

Vectorization summary

Concept	R	Python / NumPy
element-wise arithmetic	built into atomic vectors	NumPy arrays
element-wise comparisons	built into atomic vectors	NumPy arrays
logical operators	`&`, `|`, `!`	`&`, `|`, `~`
element-wise choice	`ifelse()`	`np.where()`
scalar repetition	recycling	broadcasting
unequal sizes	repeat by total length	compare shapes from the right
explicit iteration	`for`, later `lapply()` / `purrr`	comprehension, `for`
reduction	`sum()`, `mean()`, `rowMeans()`	`.sum()`, `.mean(axis=...)`

Subsetting summary

Goal	R	Python / NumPy
first element	`x[1]`	`x[0]`
last element	`x[length(x)]`	`x[-1]`
exclude first	`x[-1]`	`x[1:]`
regular range	`x[2:5]`	`x[1:5]`
arbitrary positions	`x[c(1, 3)]`	`x[[0, 2]]` (NumPy)
Boolean filter	`x[x > 0]`	`x[x > 0]` (NumPy)
all rows, second column	`x[, 2]`	`x[:, 1]`
independent copy	automatic (copy-on-modify)	`x.copy()` (NumPy)

Takeaways

- R atomic vectors and NumPy arrays support element-wise operations; Python lists require explicit iteration or comprehensions.
- R indexes from 1 and uses negative indexes for exclusion; Python indexes from 0, counts backward with negative indexes, and uses half-open slices.
- R's `[` supports integer, logical, and name-based selection. NumPy adds integer-array and Boolean indexing to Python's usual indexing syntax.
- Basic NumPy slices are always views; advanced indexing is a copy. Make copying intentional when the result will be modified.
- R recycling is based on vector length; NumPy broadcasting is based on compatible shapes. When in doubt, inspect `.shape`.